# EXPERIMENT 08

**Aim:** Design and Train a Convolutional Neural Network (CNN)

## 1. Dataset Source

**Dataset Name:** MNIST Handwritten Digit Dataset
**Source:**
The dataset is accessed using TensorFlow built-in API, which internally downloads data from Yann LeCun's MNIST database. https://www.kaggle.com/datasets/oddrationale/mnist-n-csv

## 2. Dataset Description

The MNIST dataset is a widely used benchmark dataset for image classification.

**Dataset Details:**

- Total Images: 70,000
- Training Images: 60,000
- Testing Images: 10,000
- Image Size: 28 × 28 pixels
- Channels: 1 (Grayscale)
- Number of Classes: 10 (digits 0–9)

**Target Variable:**Digit label (0–9)

**Data Representation:**Each image is a **28×28 matrix of pixel values**

- Pixel values range from **0 to 255**
- Values are normalized to **0–1**

## 3. Mathematical Formulation of CNN

CNN consists of multiple layers:

$$\boxed{Feature\ Map = (Input * Kernel) + Bias}$$

**Convolution Layer : Kernel (filter) extracts features such as edges and shapes**
**Activation Function (ReLU)**

$$\boxed{f(x) = max(0, x)}$$

- Removes negative values
- Adds non-linearity

**Pooling Layer (Max Pooling)**

$$Output = max(region)$$

- Reduces size of feature maps
- Retains important features

**Fully Connected Layer**

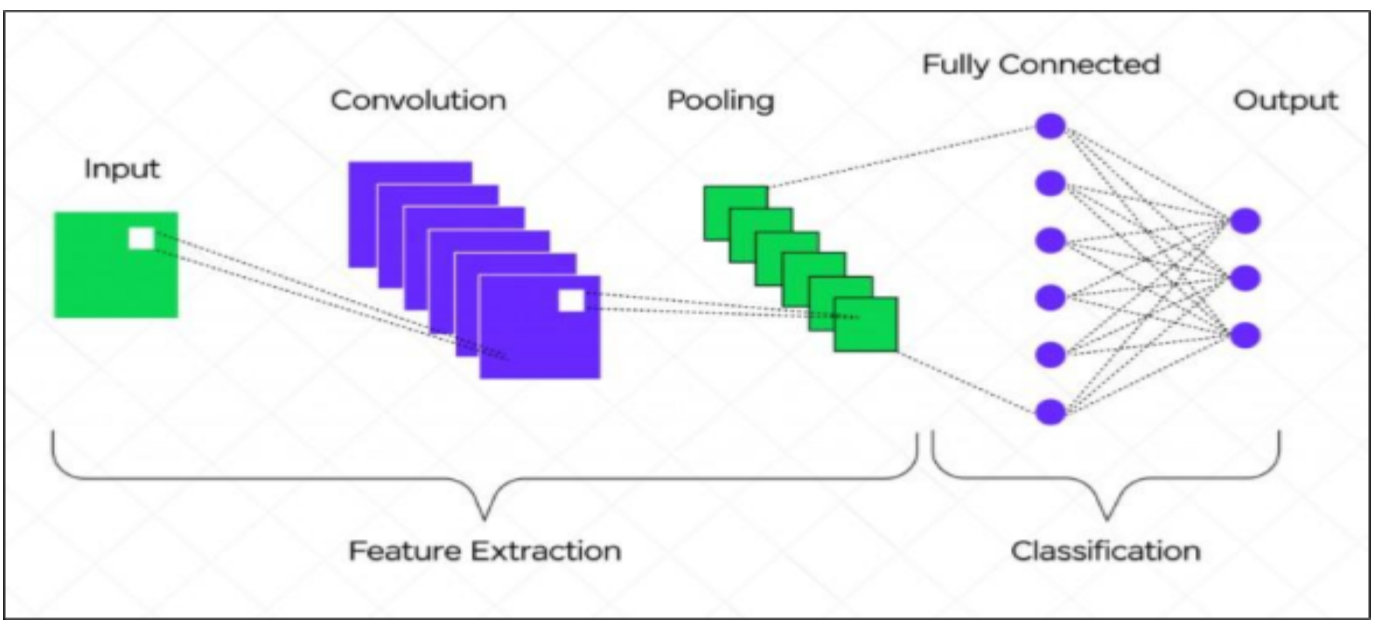
$$Z = W.X + b$$

**Softmax Output:**
Produces probability distribution

$$Softmax(z_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

**Loss Function:Categorical Crossentropy:**

$$Loss = -\sum y \log(\hat{y})$$



## 4.Algorithm Limitations of CNN

Although Convolutional Neural Networks (CNNs) are highly effective for image processing tasks, they have several limitations:

**1. High Computational Cost**

- Requires heavy matrix calculations
- Needs powerful hardware (GPU/TPU)
- Execution on normal systems can be slow

**2. Requires Large Dataset**

- Small datasets lead to poor learning
- Accuracy decreases with limited data

**3. Overfitting Problem**

- Model may memorize training data
- High training accuracy but low testing accuracy

**Solutions:**

- Dropout
- Data augmentation
- Regularization

**4. Complex Architecture Design**

- Difficult to choose:
    - Number of layers
    - Filter sizes
    - Number of neurons

**5. Time-Consuming Training**

- Large datasets and multiple epochs increase training time

**6. Sensitive to Hyperparameters**

- Performance depends on:
    - Learning rate
    - Batch size
    - Number of filters
    - Epochs

**7. Requires Data Preprocessing**

- Data must be:
    - Normalized
    - Reshaped
    - Properly formatted

**<u>5. Methodology / Workflow:</u>**

**Step 1: Dataset Loading**

- Load MNIST dataset using TensorFlow
- Obtain:
    - Training data (X_train, y_train)
    - Testing data (X_test, y_test)

**Step 2: Data Preprocessing**

- Normalize pixel values (0–255 → 0–1)
- Reshape images into 4D format
- Apply one-hot encoding to labels

**Step 3: Model Building**

CNN architecture includes:

- Convolution layers
- ReLU activation
- Pooling layers
- Flatten layer
- Dense layer
- Output layer (Softmax)

**Step 4: Model Compilation**

- Optimizer: Adam
- Loss Function: Categorical Crossentropy
- Metric: Accuracy

**Step 5: Model Training**

- Train using multiple epochs
- Use batch processing
- Validate during training

**Step 6: Model Evaluation**

- Evaluate on test data
- Measure accuracy and loss

**Step 7: Visualization**

- Plot:
    - Accuracy vs Epochs
    - Loss vs Epochs

**<u>Workflow Summary:</u>**

**Images → Preprocessing → CNN Layers → Training → Evaluation → Results**

## <u>6. Performance Analysis:</u>

### 1. Accuracy

- Measures correct predictions
- Range: 0 to 1
- Higher value indicates better performance

### 2. Loss

- Measures prediction error
- Lower value indicates better model

### Observations

- Training accuracy increases gradually
- Validation accuracy remains close to training accuracy
- Loss decreases over epochs

This indicates:

- Proper learning
- Minimal overfitting

### Result Interpretation

- High accuracy (~98–99%) indicates strong performance
- Low loss shows minimal error
- Model generalizes well to unseen data

## 7. Hyperparameter Tuning

Hyperparameters control model behavior and are set before training.

### Important Hyperparameters

| Hyperparameter | Description |
|---|---|
| Epochs | Number of training cycles |
| Batch Size | Samples per update |
| Filters | Feature detectors |
| Kernel Size | Size of convolution filter |
| Learning Rate | Speed of learning |
| Dropout Rate | Prevents overfitting |

**Tuning Process: Different values are tested:**

- Epochs: 5, 10, 15
- Batch Size: 32, 64
- Dropout: 0.3, 0.5

Best combination selected based on:

- Highest accuracy
- Lowest loss

**Impact of Tuning**

- Improves accuracy
- Reduces overfitting
- Stabilizes model
- Enhances generalization

**Tools Used**

- Manual tuning
- Trial and error

**Conclusion:**The CNN model was successfully trained and evaluated using the MNIST dataset. Proper preprocessing and hyperparameter tuning resulted in high accuracy and efficient performance in image classification tasks.